

LangChain 1.0 应该要怎么用

迈入企业级 AI 应用的工程化时代

Significant reduction in integration costs | 50%+ Development Efficiency



Based on LangChain 1.0 Stable Release

本次分享大纲



01 系统认知

核心概念、设计思路与架构升级



02 代码实操

循序渐进的三层实战：

💬 对话 (Chat)

🤖 智能体 (Agent)

💰 业务落地 (Business)



03 深度总结

学习路径与资源推荐

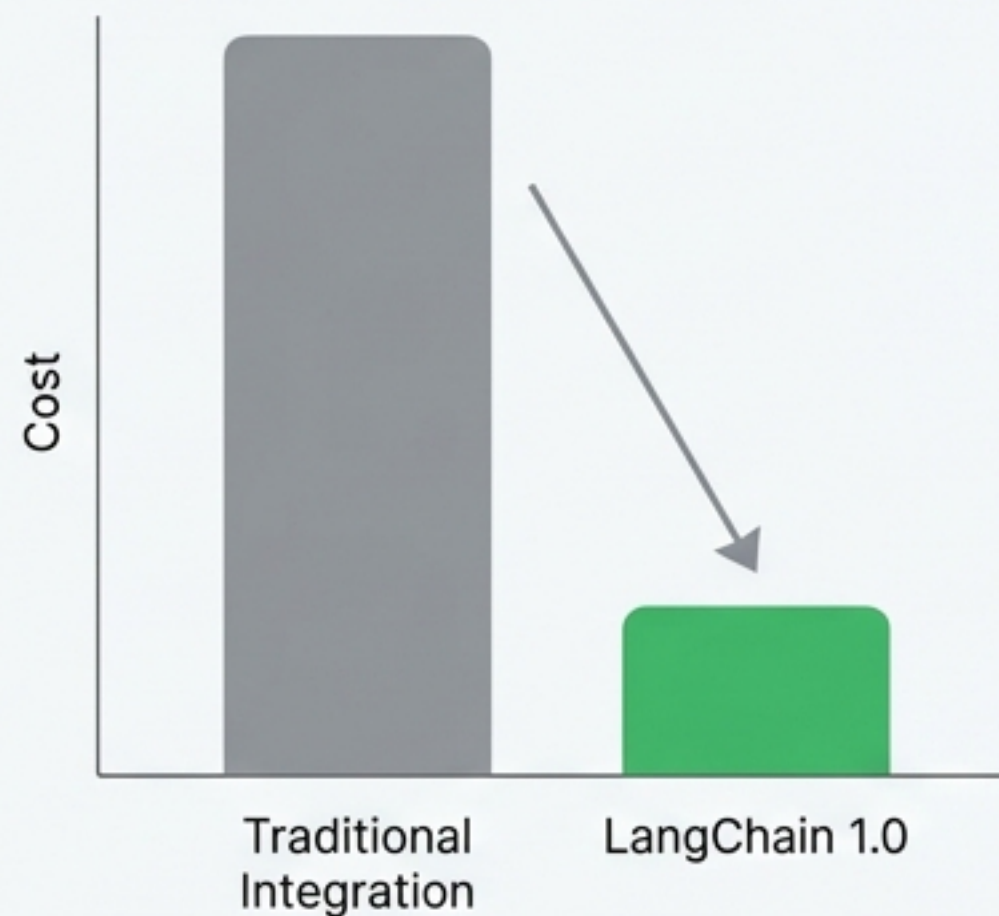
从“玩具”到“工程”的跨越

LangChain 的发布标志着大模型应用正式迈入企业级工程化时代。它解决了以往大模型开发中的碎片化、低效和难以维护的问题。

50%+

开发效率提升

通过标准化接口与模块化组件，显著降低集成与维护成本。



为什么选择 LangChain 1.0?

Version 0.3 (及以前)



拒绝打补丁

已停止更新，开发体验割裂，被称为“打补丁”式的框架。

Version 1.0 (Stable)



推倒重来，架构重构

专为生产级 AI 智能体设计，长期稳定支持。

⚠ 警示：现在谁在教你 0.3 版本，就是在浪费你的时间。

什么是 LangChain?

大模型与外部工具和数据的桥梁



让大模型不仅能思考 (Think)，还能通过调用工具去行动 (Act)。

1.0 版本的三大核心特性

轻量 (Lightweight)



架构精简，不占用大量内存和显存，运行高效。

稳定 (Stable)



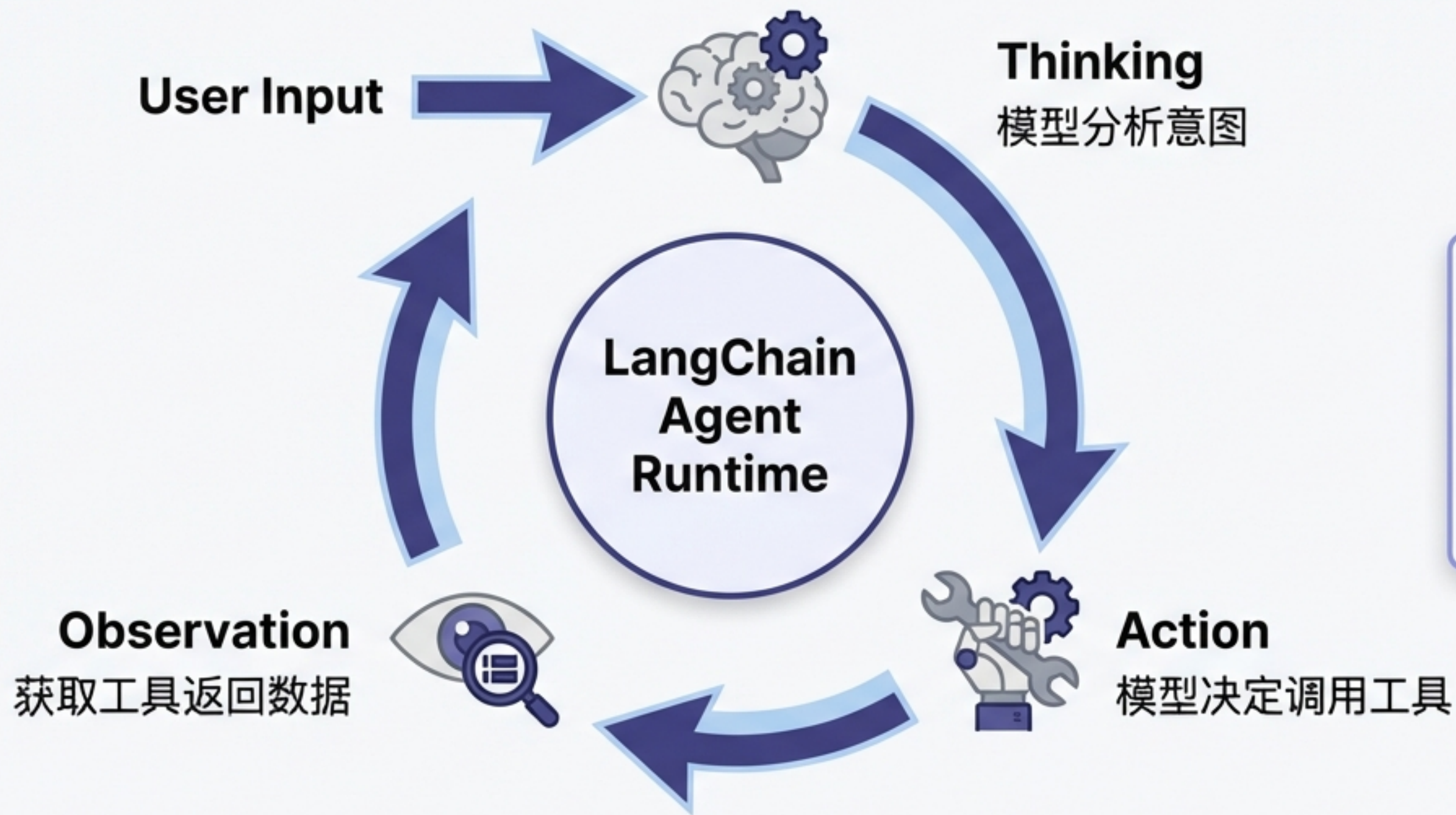
正式发布的 LTS 版本，API 接口标准化，向后兼容。

模块化 (Modular)



功能组件化设计，开箱即用，无需关心底层复杂实现。

核心概念：Agent（智能体）



赋予 LLM “手脚”，使其能够自主感知环境并执行任务。

全新的中间件系统架构

User / Application

Middleware System (中间件)

History
Compression
(压缩长对话)

Data Masking
(敏感数据脱敏)

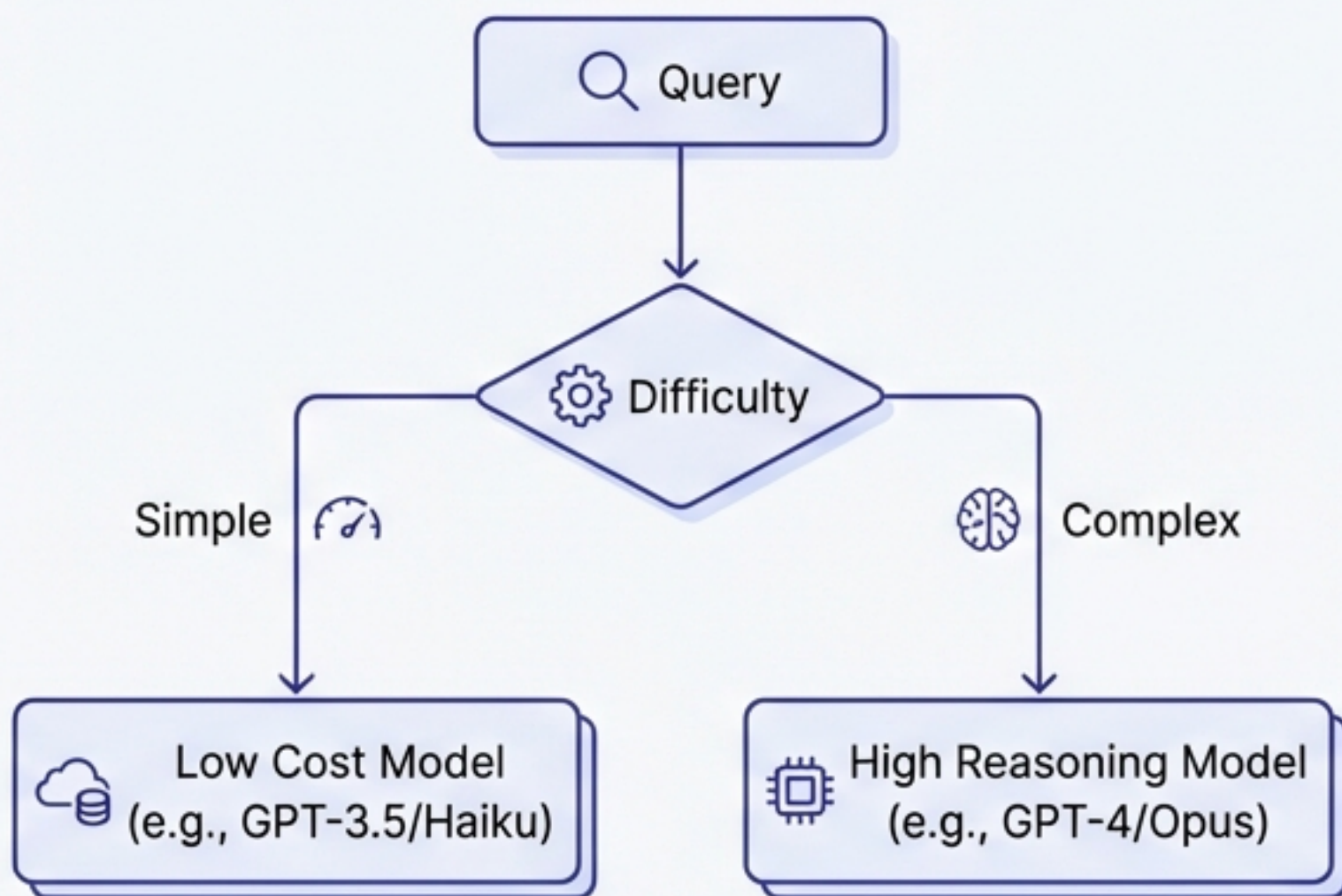
Model / Tools

自动处理 Context Window 溢出，防止“遗忘”。

企业级安全特性，
保护内部数据。

进阶架构能力

动态模型切换 (Dynamic Router)



人在回路 (Human-in-the-loop)



针对医疗、金融等高风险场景，集成人工审核机制。

实操准备：环境与工具

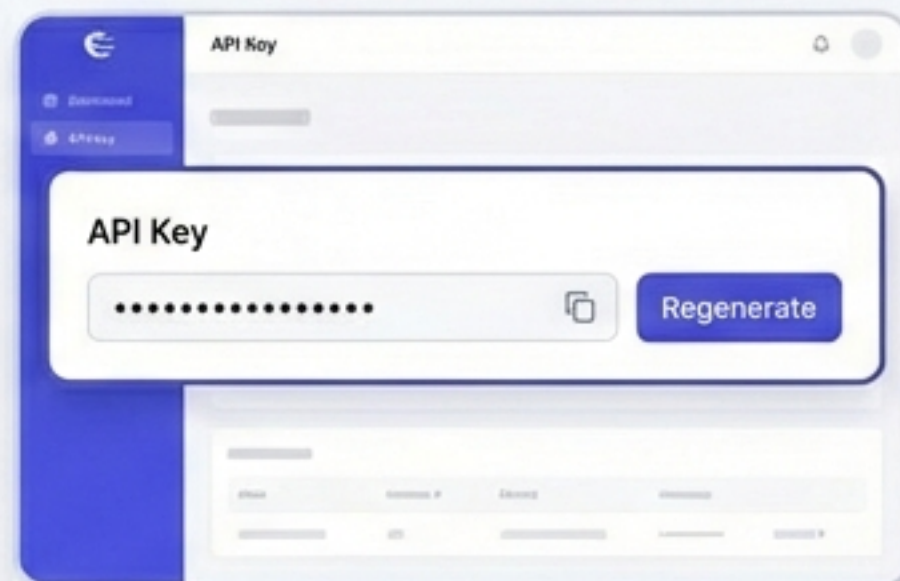
核心库

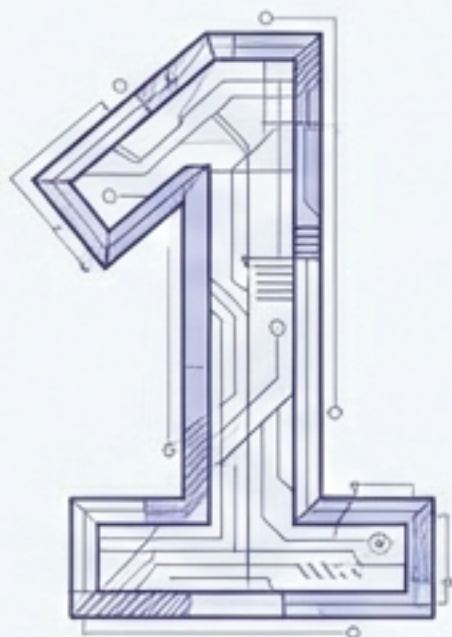
```
pip install langchain-openai
```

使用兼容 OpenAI 接口标准的通用包，灵活性极高。

推荐 API 平台 - 硅基流动 (SiliconFlow)

- 国内直连，无需代理 (No VPN needed)
- 注册赠送额度 (Free Credits)
- 兼容 DeepSeek, Qwen, Yi 等多款模型

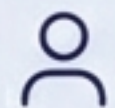




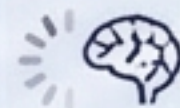
Level 1: 初次对话

The 'Hello World' of AI Engineering

目标：建立与大模型的基础连接。即使是复杂的 Agent，也是基于最简单的 `invoke`（调用）开始的。



User: 请告诉我美国有几个州



AI: ...

Level 1: 代码实现



```
from langchain_openai import ChatOpenAI
```

1. 初始化模型

```
llm = ChatOpenAI(  
    base_url="https://api.siliconflow.cn/v1",  
    api_key="sk-...",  
    model="deepseek-ai/DeepSeek-V3"  
)
```

标准化模型实例

2. 发起直接调用

```
response = llm.invoke("请告诉我美国有几个州")  
print(response.content)
```

同步调用方法

Level 1: 结果分析



> 美国共有 50 个州，分别是：阿拉巴马州、阿拉斯加州.....

纯大脑模式 (Brain Only)

模型仅依靠预训练的内部知识 (Weights) 进行回答。

缺点：无法获取实时信息，无法执行外部操作。

Next Step: 赋予它工具。

Level 2: 赋予模型“手脚”

Agents & Tools

2



我们通过 `create_react_agent` 将模型与工具绑定。模型将学会"思考"何时该使用工具。

Level 2: 定义工具 (Tools)

Tool Definition

```
from langchain_core.tools import tool

@tool
def get_weather(location: str):
    """获取指定位置的当前天气情况"""
    # 模拟返回数据
    return f"{location} 的天气是晴朗, 温度 25°C"

@tool
def search_web(query: str):
    """模拟搜索引擎"""
    return "搜索结果..."

tools = [get_weather, search_web]
```

装饰器将普通 Python 函数转化为 LangChain 工具

这是给 AI 看的说明书, AI 依此决定是否调用

Level 2: 初始化智能体 (Agent)

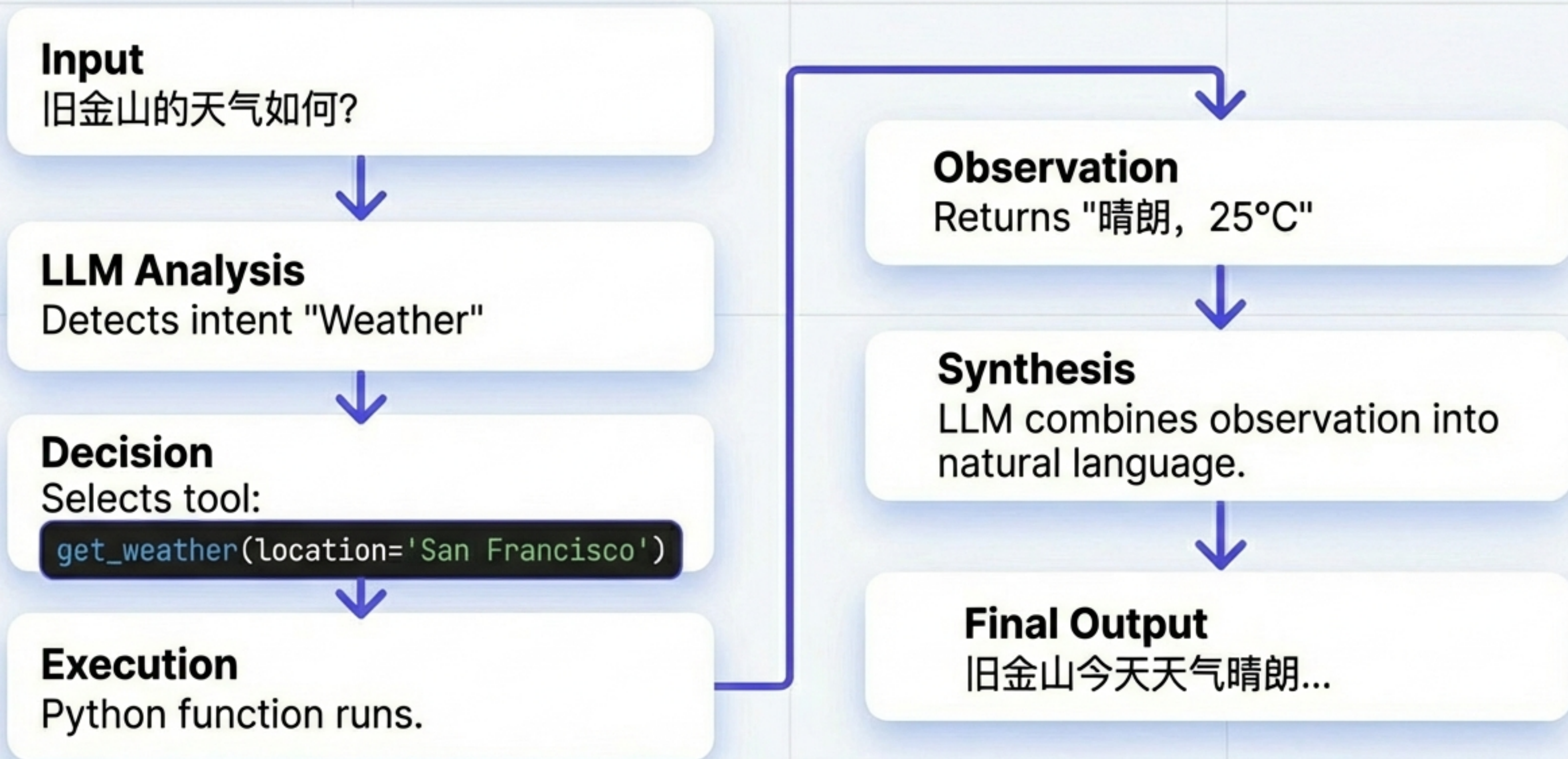
```
from langgraph.prebuilt import create_react_agent
# 将模型与工具绑定
agent_executor = create_react_agent(llm, tools)

# 执行任务
response = agent_executor.invoke({
    "messages": [("user", "旧金山的天气如何?")]
})
```

create_react_agent: 这是 LangChain 1.0 (LangGraph) 推荐的构建方式。

它自动处理了 Prompt 注入、工具解析和循环执行的逻辑。

Level 2: 执行逻辑 (The Loop)



Level 2: 代理的思考过程

[Agent] Starting...

[LLM] Thought: 用户想知道天气, 我应该调用 get_weather 工具。

[Tool] Invoking: get_weather({'location': 'San Francisco'})

[Tool] Result: 晴朗, 72度

[LLM] Final Answer: 旧金山的天气是晴朗...

💡 **白盒化体验:** 我们可以完整看到 AI 是如何一步步解决问题的。

Level 3: 业务落地

Real-world Business Scenario

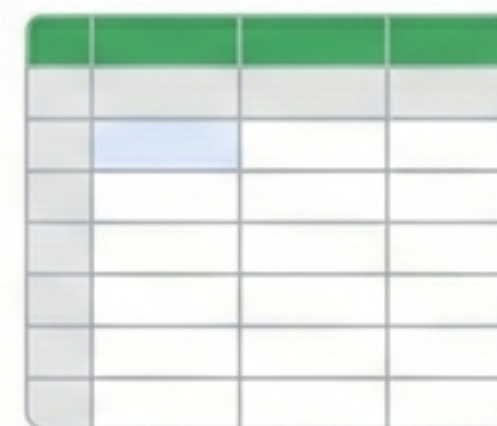
3



Unstructured Data



Agent
Processing



Structured CSV

不仅仅是聊天。我们要让 Agent 执行 I/O 操作，将非结构化数据清洗并持久化到本地文件系统。

Level 3: 自定义业务工具

```
import csv

@tool
def save_to_csv(data_list: list):
    """
    将字典列表保存为 CSV 文件。
    参数 data_list: 包含数据的字典列表
    """
    keys = data_list[0].keys()
    with open('output.csv', 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=keys)
        writer.writeheader()
        writer.writerows(data_list)
    return "文件保存成功"
```

Tool Definition

这个工具不仅是计算，而是产生了实际的副作用（Side Effect）——在磁盘上创建文件。